P2447R5

`std::span` over an initializer list

Published Proposal, 2023-09-12

Authors:

Arthur O'Dwyer

Federico Kircheis

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:

7

Abstract

`span<const int>` can be a lightweight drop-in replacement for `const vector<int>&` in the same way that `string_view` can replace `const string&`. While `"abc"` binds to a `string_view` function parameter, `{1,2,3}` fails to bind to a `span<const int>` function parameter. We show why this gap is undesirable, and propose to close it, ideally as a DR.

Table of Contents

Informative References

§ 1. Changelog

- R5 (post-LEWG 2023):
  - Reintroduce feature-test macro `__cpp_lib_span_initializer_list`.
  - Add Annex C entries to § 6 Proposed wording; LEWG points out that it's easy for LWG to eliminate insertions they deem redundant.
- R4 (pre-Varna 2023):
  - Reorganize references to [P2752] and fix HTML goofs in proposed wording.
- R3:
  - Changed primary authorship from Federico Kircheis to Arthur O'Dwyer.
  - Removed R2's feature-test macro `__cpp_lib_span_init`; it didn't seem motivated.
- R2:
  - Discussed in LEWG telecon, 2022-07-26

§ 2. Background

C++17 added `string_view` as a "view" over constant string data. Its main purpose is as a lightweight drop-in replacement for `const string&` function parameters.

C++14 `string`	C++17 `string_view`

`int take(const std::string& s) {`
`  return s[0] + s.size();`
`}`
	`int take(std::string_view sv) {`
	`  return s[0] + s.size();`
	`}`

`std::string abc = "abc";` `take(abc);`	`std::string abc = "abc";` `take(abc);`
`take("abc");`	`take("abc");`
`take(std::string("abc"));`	`take(std::string("abc"));` `take(std::string_view("abc"));`

C++20 added `span<const T>` as a "view" over constant contiguous data of type `T` (such as arrays and vectors). One of its main purposes (although not its only one) is as a lightweight drop-in replacement for `const vector<T>&` function parameters.

C++17 `vector`	C++20 `span`

`int take(const std::vector<int>& v) {`
`  return v[0] + v.size();`
`}`
	`int take(std::span<const int> v) {`
	`  return v[0] + v.size();`
	`}`

`std::vector<int> abc = {1,2,3};` `take(abc);`	`std::vector<int> abc = {1,2,3};` `take(abc);`
`take({1,2,3});`	

`take({}); // size=0`
`take({{1,2,3}});`
`take(std::vector{1,2,3});`
`take(std::initializer_list<int>{1,2,3});`
	`take({}); // size=0`
	`take({{1,2,3}});`
	`take(std::vector{1,2,3});`
	`take(std::initializer_list<int>{1,2,3});`
	`take(std::span<const int>({1,2,3}));`

This table has a conspicuous gap. The singly-braced initializer list `{1,2,3}` is implicitly convertible to `std::vector<int>`, but not to `std::span<const int>`.

§ 3. Solution

We propose simply that `std::span<const T>` should be convertible from an appropriate *braced-initializer-list*. In practice this means adding a constructor from `std::initializer_list`.

§ 3.1. Implementation experience

This proposal has been implemented in Arthur's fork of libc++ since October 2021. See "`span` should have a converting constructor from `initializer_list`" (2021-10-03) and [Patch].

§ 3.2. What about dangling?

`span`, like `string_view`, is specifically designed to bind to rvalues as well as lvalues. This is what lets us write useful code like:

```
int take(std::string_view s);
std::string give_string();
int x = take(give_string());

int take(std::span<const int> v);
std::vector<int> give_vector();
int x = take(give_vector());
```

Careless misuse of `string_view` and `span` outside a function parameter list can dangle:

```
std::string_view s = give_string(); // dangles
std::span<const int> v = give_vector(); // dangles
```

P2447 doesn't propose to increase the risk in this area; dangling is already likely when `span` or `string_view` is carelessly misused. We simply propose to close the ergonomic syntax gap between `span` and `string_view`.

Before	After P2447

`std::string_view s = "abc"; // OK`
`std::string_view s = "abc"s; // dangles`
`std::span<const char> v = "abc"; // OK`
`std::span<const char> v = "abc"s; // dangles`
	`std::string_view s = "abc"; // OK`
	`std::string_view s = "abc"s; // dangles`
	`std::span<const char> v = "abc"; // OK`
	`std::span<const char> v = "abc"s; // dangles`

`std::span<const int> v = std::vector{1,2,3}; // dangles`
`auto v = std::span<const int>({1,2,3}); // dangles`
`std::span<const int> v = {{1,2,3}}; // dangles`
	`std::span<const int> v = std::vector{1,2,3}; // dangle`
	`auto v = std::span<const int>({1,2,3}); // dangle`
	`std::span<const int> v = {{1,2,3}}; // dangle`
	`std::span<const int> v = {1,2,3}; // dangle`

3.3. Why not just double the braces?

Since we can already write

```
std::span<const int> v = {{1,2,3}}; // dangles
```

then why not call that "good enough"? Why do we need to be able to use a single set of braces?

Well, a single set of braces is good enough for `vector`, and we want `span` to be a drop-in replacement for `vector` in function parameter lists, so we need to support the syntax `vector` does. There was a period right after C++11 where some people were writing

```
std::vector<int> v = {{1,2,3}};
```

but by C++14 we had settled firmly on "one set of braces" as the preferred style (matching the preferred style for C arrays, pairs, tuples, etc.)

So I prefer to turn the question around and say: Since we can already implicitly treat `{{1,2,3}}` as a `span`, how could there be any additional harm in treating `{1,2,3}` as a `span`?

3.3.1. Better performance via synergy with P2752

[P2752], adopted as a DR at Varna 2023, allows a constant `initializer_list` like `{1,2,3}` to refer to a backing array in static storage, rather than forcing all backing arrays onto the stack. This doesn't change anything dangling-wise: referring to the backing array of an `initializer_list` outside that `initializer_list`'s lifetime remains undefined behavior.

```
std::string_view s = "abc";            // OK, no dangling
std::span<const int> v1 = {1,2,3};     // dangles, even after P2752
```

Today, `{{1,2,3}}` converts to `span` by materializing a temporary `const int[3]` on the stack. Tomorrow, if P2447 is adopted, `{1,2,3}` will convert to `span` via an `initializer_list` that refers to a backing array in static storage. In other words, the `initializer_list` constructor we propose here in P2447 is "more optimizer-friendly" than today's array-temporary constructor.

This example (Godbolt) shows how P2447 lets us benefit from P2752's optimization:

```
int perf(std::span<const int>);

int test() {
    return perf({{1,2,3}});
}
```

	`{{1,2,3}}`	`{1,2,3}`
Before 2752	Array on stack	Ill-formed
Today	Array on stack	Ill-formed
P2447	IL in rodata, tail-call	IL in rodata, tail-call

In each row, there's no performance difference between the single-braced or double-braced form. But the only way to reach the bottom row (tail-call, no stack usage) in *either* column is to adopt P2447, which by a happy coincidence also permits the single-braced form.

§ 4. Breaking changes

This change will, of course, break some code (most of it pathological). We propose adding three new examples to Annex C. But *any* change to overload sets can break code, and sometimes LWG doesn't bother with an Annex C entry. For example, C++23 adopted [P1425] "Iterator-pair constructors for `stack` and `queue`" with no change to Annex C, despite its breaking code like this:

```
void zero(queue<int>);
void zero(pair<int*,int*>);
int a[10];
void test() { zero({a, a+10}); }
```

Before: Calls `zero(pair<int, int>)`.
After P1425: Ambiguous.
To fix: Eliminate the ambiguous overloading, or cast the argument to `pair`.

Therefore, we're happy for LWG to eliminate any or all of our proposed Annex C entries if they're going too far into the weeds.

For explanation and suggested fixits for each of the Annex C examples included in § 6 Proposed wording, see P2447R4 §4.

§ 5. Straw polls

P2447R4 was presented to LEWG on 2023-09-12. The following polls were taken. The first was classified as "no consensus," the second as "weak consensus."

	SF	***F***	***N***	***A***	***SA***
Forward P2447R4 to LWG for C++26 and as a defect.	*2*	*5*	*3*	*2*	*1*
Forward P2447R4 to LWG for C++26 (not as a defect).	*2*	*6*	*4*	*1*	*1*

§ 6. Proposed wording

Modify [span.syn] as follows:

```
#include <initializer_list>     // see [initializer.list.syn]
```

Modify [span.overview] as follows:

```
  template<size_t N>
    constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
  template<class T, size_t N>
    constexpr span(array<T, N>& arr) noexcept;
  template<class T, size_t N>
    constexpr span(const array<T, N>& arr) noexcept;
  template<class R>
    constexpr explicit(extent != dynamic_extent) span(R&& r);


  constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;
  constexpr span(const span& other) noexcept = default;
  template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

Modify [span.cons] as follows:

```
constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;
```

Constraints: `is_const_v<element_type>` is `true`.

Preconditions: If `extent` is not equal to `dynamic_extent`, then `il.size()` is equal to `extent`.

Effects: Initializes *`data_`* with `il.begin()` and *`size_`* with `il.size()`.

Modify [diff.cpp26] as follows:

> NOTE: For explanation and suggested fixits for each of these examples, see P2447R4 §4. My understanding is that Annex C wording shouldn't contain that extra material.

[containers]: containers library

1. Affected subclause: [span.overview]
Change: `span<const T>` is constructible from `initializer_list<T>`.
Rationale: Permit passing a braced initializer list to a function taking `span`.
Effect on original feature: Valid C++ 2023 code that relies on the lack of this constructor may refuse to compile, or change behavior. For example:

```
void one(pair<int, int>);    // #1
void one(span<const int>);   // #2
void t1() { one({1,2}); }  // ambiguous between #1 and #2; previously called #1

void two(span<const int, 2>);
void t2() { two({{1,2}}); }  // ill-formed; previously well-formed

void *a[10];
int x = span<void* const>{a, 0}.size(); // x is 2; previously 0
any b[10];
int y = span<const any>{b, b+10}.size(); // y is 2; previously 10
```

Add a feature-test macro to [version.syn]/2 as follows:

```
#define __cpp_lib_span                    202002L // also in <span>
#define __cpp_lib_span_initializer_list   XXYYZZL // also in <span>
#define __cpp_lib_spanstream              202106L // also in <spanstream>
```

§ 7. Acknowledgments

- Thanks to Federico Kircheis for writing the first drafts of this paper.
- Thanks to Jarrad Waterloo for his support.

§ References

§ Informative References

[P1425]
Corentin Jabot. *Iterator-pair constructors for stack and queue*. March 2021. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1425r4.pdf

[P2752]
Arthur O'Dwyer. *Static storage for braced initializers*. June 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html

[Patch]
Arthur O'Dwyer. *Implement P2447 std::span convertible from std::initializer_list*. October 2021. URL: https://github.com/Quuxplusone/llvm-project/pull/17